



7

17

3060 sp

نظري

كلية الهندسة المعلوماتية

السنة الثانية

# Multithreaded programming

د. غاندي هسام



برمجة ٣

RB Informatics; 20 / 11 / 2025

## Multithreaded programming

الفرق بين ال Multithreading و Multiprocessing :

- مفهوم تعدد المهام : عندما تستخدم هاتفك او حاسوبك ترى انه يمكنك تشغيل عدة برامج في وقت واحد كل برنامج شغال في الذاكرة يعتبر process مثلا اذا قمت بتشغيل خمسة برامج مع بعض فهذا يعني ان نظام التشغيل ينظم عمل خمسة processes مع بعض الية تشغيل عدة برامج مع بعض تسمى Multiprocessing
- من جهة أخرى في البرنامج الواحد يمكنك تنفيذ عدة أوامر مع بعض وجعل المستخدم يشعر كأنها تنفذ في وقت واحد مثلا في حال كنت تلعب لعبة مثل كرة القدم تجد انه هناك عدة أشياء تحدث في وقت واحد كسماع الأغاني أصوات المشجعين صوت الحكم هذه الالية تسمى Multithreading

### بالمختصر:

ال process تعني برنامج قيد التنفيذ وقد قام نظام التشغيل بحجز مساحة خاصة له في الذاكرة وتخصيص موارد له  
ال Thread عبارة عن مجموعة أوامر يتم تنفيذها اثناء تنفيذ أوامر أخرى في نفس البرنامج أي ال thread محتوى داخل ال process  
كل عملية (أي برنامج قيد التنفيذ ) تبدأ دائما بخيط واحد رئيسي على الأقل يمكن للعملية الواحدة ان تحتوي على خيوط متعددة تعمل بشكل مستقل لكنها تشترك في نفس مساحة الذاكرة الخاصة للعملية الام هذا يسمح لها بالتواصل وتبادل البيانات بسهولة

### الهدف من ال Thread :

1. التوازي او التزامن : يسمح بتقسيم مهمة كبيرة الى مهام اصغر (خيوط)يمكن ان تعمل بنفس الوقت .
2. كتابة برنامج ذات كفاءة عالية : لأنه يسمح للمبرمجين باستغلال وقت الخمول (Idle Time) الموجود في معظم البرامج وقت الخمول هو عندما يقوم البرنامج بعمل يتطلب انتظار مثل قراءة ملف من القرص الصلب او انتظار استجابة من الشبكة فان الخيط بأكمله يتوقف عن العمل ويانتظر اكمال عملية الادخال والإخراج هذا الوقت الضائع هو وقت خمول لا يتم فيه استغلال المعالج في حال تعدد الخيوط عندما ينتظر خيط معين Thread A اكتمال عملية الادخال والإخراج يستطيع نظام التشغيل ان يحول المعالج فورا الى خيط اخر B

Thread داخل نفس البرنامج بهذه الطريقة يستمر البرنامج في انجاز مهام أخرى بدلا من التوقف والانتظار مما يؤدي الى استجابة اسرع واستغلال افضل للموارد .

## البنات الأساسية لل Multithreading :

1. كلاس Thread : كلاس جاهز في جافا موجود في حزمة java.lang تمثل الخيط الفعلي في نظام التشغيل أي كائن يتم إنشاؤه من هذه الفئة او يرثها يمثل مسار تنفيذ جديد منفصل عن الخيط الرئيسي حيث توفر جميع الطرق اللازمة لإدارة الخيط والتحكم فيه مثل (البدا،الإيقاف المؤقت،الانتظار)
2. واجهة التشغيل (Runnable Interface) :  
هي واجهة جاهزة في جافا موجودة في حزمة java.lang تمثل المهمة او الكود الذي سيتم تنفيذه بواسطة الخيط (Thread) هي لاتمثل الخيط نفسه بل ما سيفعله الخيط تحتوي هذه الواجهة على دالة وحيدة مجردة يجب تطبيقها وهي `public void run();`  
التعليمات البرمجية التي تضعها داخل هذه الطريقة هي التي سيتم تنفيذها عندما يبدأ الخيط عمله.

### الطرق التي يوفرها كلاس Thread :

- فئة Thread ليست مجرد حاوية للمهمة بل هي مدير الخيط التي توفر الطرق الأساسية للتحكم في دورة حياة الخيط 7
- `void start()` : عندما تقوم بإنشاء كائن من فئة خيط فانه يكون في حالة جديد `new()` استدعاء `start()` هو الذي يطلب من نظام التشغيل تخصيص الموارد الضرورية وإنشاء مسار تنفيذي منفصل ينقل الخيط من حالة جديد الى حالة قابل للتشغيل يتم استدعاء طريقة `run()` الخاصة بالكائن تلقائيا بواسطة آلة جافا الافتراضية
- يجب استدعاء `start()` مرة واحدة فقط لكل كائن خيط اذا استدعيته مرتين سيحدث خطأ `IllegalThreadStateException`
- `void run()` : تحتوي على الكود الفعلي الذي سيقوم الخيط بتنفيذه
- لا يجب على المبرمج ابدا استدعاء `run()` بشكل مباشر اذا قمت باستدعائها فسيتم تنفيذ الكود في نفس الخيط الحالي وتكون قد فشلت في انشاء خيط جديد
- `final void join()` :توقف الخيط الحالي حتى ينتهي الخيط الذي تم استدعاء `join()` عليه من عمله بالكامل
- `static void sleep(long ms)` : تعلق الخيط الحالي وتجعله ينام لفترة زمنية محددة
- `final boolean isAlive()` : تتحقق مما اذا كان الخيط قيد التشغيل ترجع `true` اذا كان الخيط قد بدأ تشغيله أي تم استدعاء `start` ولم ينته بعد
- ترجع `false` اذا كان الخيط في حالة جديد `new()` قبل استدعاء `start()` او في حالة `Terminated` (ميت) بعد انتهاء `run()`
- `final String getName()` : تعيد اسم الخيط تفيد في تتبع الأخطاء
- `final int getPriority()` : تعيد الرقم الذي يحدد أولوية الخيط بالنسبة للخيط الأخرى

■ **ملاحظة:** لا تضمن جافا انه يتم تنفيذ الخيط ذي الأولوية الأعلى أولا بالضرورة لكنها قد تعد تلميحا لنظام التشغيل سنستخدم طرق افضل لتحديد الأولويات نتحدث عنها في الفقرات القادمة

### طرق انشاء ال Thread :

1. **الوراثة من فئة Thread:** تعتبر هذه الطريقة هي الابطسط للبدء حيث يتم دمج المهمة وآلية التحكم في الخيط في كائن واحد

الخطوات :

1. ننشئ فئة جديدة ترث من فئة Thread

2. إعطاء اسم للخيط من خلال الباني بدلا من الاسم الافتراضي الذي تعطيه جافا حيث يتم استدعاء منشئ الاب الذي يحوي باني يقبل سلسلة نصية كمعامل ويستخدمها لتعيين اسم الخيط

3. نقوم بتجاوز (override) الدالة run() أي كود ستضعه هنا سيتم تنفيذه في الخيط المنفصل

4.

5. في الخيط الرئيسي قم بانشاء كائن من هذه الفئة ثم استدع طريقة start() عليه

```
// 1. Define the thread class
class MyThread extends Thread {
    MyThread(String name) {
        super(name); // Name the thread
    }

    // Entry point of the thread
    public void run() {
        System.out.println(getName() + " is running inside the
thread.");
    }
}

// 2. Separate class to run the program
class ExtendThread {
    public static void main(String[] args) {
        System.out.println("Main thread starting.");
        MyThread mt = new MyThread("Child #1");

        mt.start();
    }
}
```



## 2- تطبيق لواجهة Runnable :

تتطلب هذه الطريقة ثلاث خطوات رئيسية

**تعريف المهمة :** نقوم بإنشاء كلاس ونعلن انه ينفذ الواجهة Runnable ونقوم بتجاوز الطريقة الوحيدة التي تفرضها الواجهة وهي `public void run()` نضع داخل جسم هذه الدالة جميع الأوامر والعمليات التي نريد ان تنفذ في المسار المستقل في نهاية هذه الخطوة نكون قد انشأنا مخطط للعمل لكننا لم ننشئ الخيط بعد

**تغليف المهمة** ضمن كيان الخيط : نقوم بإنشاء كائن من الكلاس الذي انشأناه في الخطوة الأولى ثم ننشئ كائن من فئة Thread ونستخدم منشئ فئة Thread الذي يقبل كائن من نوع Runnable كمعامل

**بدء التنفيذ :** استدعاء الدالة `start()` على كائن ال Thread ثم يبدأ تنفيذ الكود الموجود في `run()` بشكل متوازي او متزامن مع الخيط الرئيسي الذي قام باستدعاء `start()`

```
// Create a thread by implementing Runnable.
class MyThread implements Runnable {
    String thrdName;
    MyThread(String name) {
        thrdName = name;
    }
    // Entry point of thread.
    public void run() {
        System.out.println(thrdName + " starting.");
        try {
            for(int count=0; count < 10; count++) {
                Thread.sleep(400);
                System.out.println("In " + thrdName +
                    ", count is " + count);
            }
        } catch (InterruptedException exc) {
            System.out.println(thrdName + " interrupted.");
        }
        System.out.println(thrdName + " terminating.");
    }
}

public static void main(String[] args) {
    System.out.println("Main thread starting.");
}
```

```
// First, construct a MyThread object.
MyThread mt = new MyThread("Child #1");

// Next, construct a thread from that object.
Thread newThrd = new Thread(mt);
// Finally, start execution of the thread.
newThrd.start();
for(int i=0; i<50; i++) {
    System.out.print(".");
    try {
        Thread.sleep(100);
    }
    catch(InterruptedException exc) {
        System.out.println("Main thread interrupted.");
    }
}
System.out.println("Main thread ending.");
}
```

### شرح الكود:

اعلان الكلاس الذي يطبق الواجهة Runnable وتعريف متغير خاص بالخيـط thrdName لتخزين اسمه انشاء باني يستقبل اسم name ويخزنه في المتغير الخاص بالخيـط thrdName هذا يسمح بتسمية الخيـط لتتبع الأخطاء في حال حدوثها

الدالة run() هي نقطة الدخول للمهمة حيث يتم طباعة رسالة بداية الخيـط للتتبع

يتم وضع الكود في كتلة try catch للتعامل مع استثناء InterruptedException وهو ضروري لان الدالة Thread.sleep() قد تقاطع

حلقة تكرارية بسيطة تتكرر 10 مرات

Thread.sleep(400) تجعل الخيـط ينام لمدة 400 ملي ثانية هذا التعليق يسمح للخيوط الأخرى بها في ذلك الخيـط الرئيسي بالحصول على المعالج والعمل

طباعة القيمة الحالية للعداد

طباعة رسالة انتهاء الخيـط بعد اكتمال الحلقة

في التابع الرئيسي main:

طباعة رسالة بداية الخيط الرئيسي

انشاء كائن من كلاس MyThread ثم انشاء كائن Thread وتحرير الكائن الذي انشأناه في الخطوة السابقة بدء تنفيذ الخيط

حلقة تكرارية للخيط الرئيسي تتكرر 50 مرة

الخيط الرئيسي يطبع نقطة في كل تكرار الخيط الرئيسي ينام لمدة 100 ميلي ثانية في كل تكرار هذا يسمح للخيط العامل child#1 بالحصول على المعالج وتنفيذ خطواته في نفس الوقت الخيط الرئيسي ينهي عمله

**ملاحظة:** يمكن ان ينتهي تنفيذ الخيط الرئيسي قبل انتهاء تنفيذ الخيط العامل

## Output

```
Main thread starting.  
Child #1 starting.  
...In Child #1, count is 0  
....In Child #1, count is 1  
....In Child #1, count is 2  
....In Child #1, count is 3  
....In Child #1, count is 4  
....In Child #1, count is 5  
....In Child #1, count is 6  
....In Child #1, count is 7  
....In Child #1, count is 8  
....In Child #1, count is 9  
Child #1 terminating.  
.....Main thread ending.
```



لان كليهما ينام لفترات زمنية فان مجدول النظام يقوم بالتبديل بينهما بشكل مستمر مما يسمح للخييط الرئيسي بالتقدم في حلقاته بينما ينتظر الخييط العامل دوره

**لاحظ** : ان تداخل رسائل الخرج يثبت ان الخييط الرئيسي لم ينتظر انتهاء الخييط العامل بعد استدعاء start() بل واصل

عمله بشكل متواز وكل منهما استغل وقت التوقف sleep الخاص بالآخر للعمل على نفس المعالج  
في الفقرات القادمة ستري طريقة افضل لضمان التزامن بشكل صحيح لا يجب الاعتماد على فترات النوم

## Static Factory method:

- طريقة المصنع الثابت تهدف الى دمج الخطوتين في عملية واحدة تبدو كأنها خطوة عمل واحدة مما يجعل الخييط جاهزا للعمل فورا
- في السياق التقليدي لانشاء الخيوط يتم عادة الفصل بين عمليتي الانشاء والبدء  

```
MyThread obj = new MyThread();
Obj.start();
```
- لكن عندما تكون النية هي البدء الفوري للمهمة بمجرد طلبها فان هذا الفصل يصبح غير ضروري تتيح لنا هذه الطريقة دمج الخطوتين في عملية واحدة مما يحقق ايجاز للكود ويقلل من فرصة الخطأ
- احدى الطرق الفعالة لتحقيق هذا الدمج هي توفير تابع مصنعي ثابت ضمن الكلاس الخاص بالخييط
- يتميز هذا التابع بالخصائص التالية :**
- ثابت:** يمكن استدعاؤه مباشرة من خلال اسم الكلاس دون الحاجة لانشاء نسخة سابقة من الكلاس
- مصنعي:** وظيفته هي انشاء كائن جديد وارجاع مرجع اليه بدلا من استخدام الباني مباشرة
- يقوم هذا التابع بتنفيذ ثلاث خطوات متسلسلة لضمان انشاء الخييط وتشغيله بشكل فوري وفعال
- انشاء نسخة جديدة من الخييط :يصبح الخييط في حالة جديد New
- استدعاء التابع start() :** يتم استدعاء التابع start() مباشرة على نسخة الخييط التي تم انشاؤها في الخطوة السابقة فيتم نقل الخييط الى حالة قابل للتشغيل ويبلغ نظام التشغيل بانه جاهز لتنفيذ التابع run()
- ارجاع مرجع للكائن المنشأ:** يرجع التابع مرجعا الى الكائن الذي تم إنشاؤه وبدء تشغيله و يصبح بإمكان البرنامج الخارجي الان الاحتفاظ بمرجع الى هذا الخييط قيد التشغيل مثلا في متغير ما بذلك يمكن استخدام هذا المرجع للتحكم بالخييط لاحقا

"Threads never sleep... well, except when you call Thread.sleep() 🤔."



```
class MyThread implements Runnable {
    Thread thrd; // A reference to the thread is stored in thrd.
    MyThread(String name) {
        thrd = new Thread(this, name); // The thread is named when it is created.
    }
    public static MyThread createAndStart(String name) {
        MyThread myThrd = new MyThread(name);
        myThrd.thrd.start(); // start the thread ← Begin executing the thread.
        return myThrd;
    }
    public void run() {
        System.out.println(thrd.getName() + " starting.");
        try {
            for(int count=0; count<10; count++) {
                Thread.sleep(400);
                System.out.println("In " + thrd.getName() + ", count is " + count);
            }
        } catch (InterruptedException exc) {
            System.out.println(thrd.getName() + " interrupted.");
        }
        System.out.println(thrd.getName() + " terminating.");
    }
}

class ThreadVariations {
    public static void main(String[] args) {
        System.out.println("Main thread starting.");
        // Create and start a thread.
        MyThread mt = MyThread.createAndStart("Child #1"); // ← Now the thread starts when it is
        created.
        for(int i=0; i < 50; i++) {
            System.out.print(".");
            try {
                Thread.sleep(100);
            } catch (InterruptedException exc) {
                System.out.println("Main thread interrupted.");
            }
        }
        System.out.println("Main thread ending.");
    }
}
```

شرح الكود:

تعريف كلاس MyThread ينفذ الواجهة Runnable حيث يتم تحديد ما سيفعله الخيط عبر التابع run()  
يتم تعريف متغير داخلي Thread thrd الذي يحمل كائن الخيط الفعلي هذا الكائن هو الذي يديره نظام التشغيل  
ويحتوي على التابع start()

: MyThread (String name)

يعتبر الباني حلقة الوصل الأساسية لإنشاء خيط جاهز للعمل حيث يقوم بدور المجمع والرابط بما ان فئة  
MyThread تمثل المهمة من خلال التابع run() ولا تمتلك القدرة على البدء الفعلي فان الباني يقوم بإنشاء الآلة  
المنفذة وهي كائن من فئة Thread وتخزينه في المتغير الداخلي thrd وتكمن أهمية الباني في استخدام الكلمة  
المفتاحية this التي تمثل مرجع الكائن الحالي الحامل للمهمة يتم تمرير هذا المرجع this الى باني الفئة thread  
وهذا الاجراء يضمن ان الخيط الجديد يعرف بدقة أين يجد التعليمات (التابع run()) التي يجب عليه تنفيذها عند  
استدعاء start()

التابع المصنعي الثابت :

يتم فيه استدعاء الباني في هذه اللحظة يتم انشاء كائن MyThread ويتم داخله انشاء وربط كائن الخيط الفعلي  
Thread thrd بالمهمة run()

يتم استدعاء التابع start على كائن الخيط الفعلي المنشأ هذا الجراء يجعل الخيط في حالة قابلية التشغيل  
يرجع التابع المرجع الى كائن MyThread الذي اصبح الان مهياً وقيد التنفيذ

التابع run() : يبدأ التابع بطباعة رسالة تشير الى بداية تنفيذ الخيط الذي تم تعيينه

يتم وضع العمليات التي قد تولد استثناءات داخل كتلة try catch

يحتوي التابع على حلقة تكرارية مصممة لتنفيذ مهمة الخيط داخل حلقة التكرار يتم استخدام التابع  
Thread.sleep(400) يتسبب هذا التابع في إيقاف الخيط الحالي مؤقتاً لمدة 400 ملي ثانية

في التابع الرئيسي main :

تبدأ عملية التنفيذ بطباعة رسالة توضح ان الخيط الرئيسي للبرنامج قد بدأ العمل

انشاء وبدء تنفيذ خيط فرعي واحد حيث يتم استدعاء التابع المصنعي الثابت

بعد انشاء الخيط الفرعي هذه الكتلة من الكود الذي تليه تمثل المهمة الخاصة للخيط الرئيسي بينما الخيط الفرعي  
يعد ويطلع من 0 الى 9 مع فترات النوم 400 ملي ثانية اما الخيط الرئيسي يقوم بتنفيذ حلقة تكرارية 50 مرة تطبع  
نقطة في كل تكرار

يستخدم الخيط الرئيسي التابع Thread.sleep(100) لإبطاء نفسه في كل تكرار

الإعلان عن انتهاء الخيط الرئيسي

■ في المثال السابق انشأنا خيط فرعي واحد ومع ذلك يمكن لبرنامجك ان يولد العديد من الخيوط حسب حاجته

```
public static void main(String[] args) {
    System.out.println("Main thread starting.");
    MyThread mt1 = MyThread.createAndStart("Child #1");
    MyThread mt2 = MyThread.createAndStart("Child #2"); // ← Create and start executing
    three threads.
    MyThread mt3 = MyThread.createAndStart("Child #3");
}
```

### سيكون الخرج

#### Output

```
Main thread starting.
Child #1 starting.
Child #2 starting.
Child #3 starting.
...In Child #3, count is 0
In Child #2, count is 0
In Child #1, count is 0
In Child #2, count is 1
In Child #3, count is 1
...In Child #1, count is 1
In Child #2, count is 2
In Child #3, count is 2
...In Child #1, count is 2
In Child #2, count is 3
In Child #3, count is 3
...In Child #1, count is 3
In Child #2, count is 4
In Child #3, count is 4
```

```
...In Child #1, count is 4
In Child #2, count is 5
In Child #3, count is 5
...In Child #1, count is 5
...In Child #3, count is 6
..In Child #2, count is 6
In Child #1, count is 6
In Child #3, count is 7
...In Child #1, count is 7
In Child #2, count is 7
....In Child #2, count is 8
In Child #1, count is 8
In Child #3, count is 8
....In Child #1, count is 9
Child #1 terminating.
In Child #2, count is 9
Child #2 terminating.
In Child #3, count is 9
Child #3 terminating.
.....Main thread ending.
```

## ■ تحديد نهاية الخيط :

■ يجب ان يستمر الخيط الرئيسي في العمل حتى تنتهي جميع الخيوط الفرعية من مهامها والا قد يغلق البرنامج قبل اكتمال العمل في الأمثلة السابقة تم إبقاء الخيط الرئيسي حيا عن طريق جعله ينام لفترة أطول من الخيوط الفرعية لكن هذا الحل غير عملي ولا يمكن تعميمه لانه لا يضمن ان الخيط الفرعي قد انتهى بالفعل

■ توفر فئة Thread آليتين رئيسيتين لتحديد ما اذا كان الخيط قد انتهى

■ التابع `isAlive()` :

■ هو تابع يستدعى على كائن الخيط يرجع قيمة منطقية `true` اذا كان الخيط لا يزال نشط او قيد التنفيذ وبرجع `false` اذا انتهى الخيط واكمل التابع `run()` عمله

■ يمكن استخدام هذا التابع في حلقة `do while` في الخيط الرئيسي لمراقبة حالة الخيط الفرعي والاستمرار في الانتظار طالما ان `isAlive()` ترجع `true`

```
class MoreThreads {
    public static void main(String[] args) {
        System.out.println("Main thread starting.");

        MyThread mt1 = MyThread.createAndStart("Child #1");
        MyThread mt2 = MyThread.createAndStart("Child #2");
        MyThread mt3 = MyThread.createAndStart("Child #3");

        do {
            System.out.print(".");
            try {
                Thread.sleep(100);
            }
            catch (InterruptedException exc) {
                System.out.println("Main thread interrupted.");
            }
        } while (mt1.thrd.isAlive() ||
                mt2.thrd.isAlive() ||
                mt3.thrd.isAlive()); // ← This waits until all threads terminate.

        System.out.println("Main thread ending.");
    }
}
```

- الفكرة هنا انه يتم استخدام التابع `isAlive()` لتحديد ما اذا كان الخيط لا يزال قيد التشغيل يستمر الخيط الرئيسي في الحلقة طالما ان أي خيط من الخيوط الفرعية لا يزال نشط
- بمجرد ان تصبح جميع الخيوط الثلاثة غير نشطة يخرج الخيط الرئيسي من حلقة الانتظار ويطبع الرسالة
- التابع `join` :
- عندما يقوم خيط ما مثل `main` باستدعاء `threadA.join` فان الخيط المستدعي يتوقف عن التنفيذ مؤقتا ولا يستأنف عمله الا بعد ان يعلن الخيط `threadA` عن انتهاء عمله بالكامل هذا يضمن التزامن الصحيح

### مثال :

```
class JoinThreads {
    public static void main(String[] args) {
        System.out.println("Main thread starting.");
        MyThread mt1 = MyThread.createAndStart("Child #1");
        MyThread mt2 = MyThread.createAndStart("Child #2");
        MyThread mt3 = MyThread.createAndStart("Child #3");
        try {
            mt1.thrd.join(); // Wait until Child #1 ends.
            System.out.println("Child #1 joined.");
            mt2.thrd.join(); // Wait until Child #2 ends.
            System.out.println("Child #2 joined.");
            mt3.thrd.join(); // Wait until Child #3 ends.
            System.out.println("Child #3 joined.");
        }
        catch (InterruptedException exc) {
            System.out.println("Main thread interrupted.");
        }
        System.out.println("Main thread ending.");
    }
}
```

Threads check their vibes with `isAlive()`  
just to prove they're still kicking



## شرح بسيط للكود:

يقوم الخيط الرئيسي بانشاء وبدء تشغيل ثلاث خيوط فرعية

يستخدم الخيط الرئيسي الدالة join() على كل خيط فرعي بالترتيب لتمثيل التأثير في ان الخيط الرئيسي يتوقف عن التنفيذ مؤقتا عند كل استدعاء ل join() هذا يضمن ان الخيط الرئيسي لا يمكنه انهاء البرنامج حتى تكون الخيوط الفرعية الثلاث قد اكملت مهمتها وانتهت

## التزامن Synchronization:

- عند استخدام الخيوط المتعددة يصبح من المهم ان يكون هناك ترتيب في العمليات وجعلها تتبع نظام محدد بدون تنسيق قد تعمل الخيوط بشكل عشوائي مما يؤدي الى نتائج غير متوقعة او خاطئة
- السبب الأكثر شيوعا للتزامن هو عندما يحتاج خيطان او اكثر الوصول الى مورد مشترك لا يمكن استخدامه الا بواسطة خيط واحد في كل مرة على سبيل المثال اذا حاول خيطان الكتابة الى نفس الملف في نفس الوقت قد يتم دمج البيانات بشكل فوضوي مما يجعل الملف تالف او غير قابل للقراءة

**الحل بالتزامن :** التزامن يضمن ان الخيط الأول يحصل على تصريح حصري للكتابة ويحتفظ بهذا التصريح حتى ينتهي من جميع عمليات الكتابة الخاصة به ثم يحرر التصريح للخيط الثاني

المفتاح الأساسي للتزامن في جافا هو مفهوم المراقب الذي يتحكم في الوصول الى كائن يعمل المراقب من خلال تطبيق مفهوم القفل

**المراقب Monitor :** كل كائن لديه مراقب داخلي مرتبط به هذا المراقب هو الآلية التي تتيح التزامن

**القفل Lock :** المراقب يعمل بناء على مفهوم القفل عندما يرغب خيط ما في الوصول الى جزء من الكود يتضمن مورد مشترك يجب عليه أولا الحصول على القفل الخاص بكائن المراقب

## مثال

```
class Line {
    synchronized public void getLine() {
        for (int i = 0; i < 3; i++) {
            System.out.println(i);
            try { Thread.sleep(100); }
            catch (Exception e) { System.out.println(e); }
        }
    }
}
```



```

class Train extends Thread {
    Line line;

    Train(Line line) { this.line = line; }
    public void run() { line.getLine(); }
}

public class Geeks {
    public static void main(String[] args) {
        Line obj = new Line();
        Train t1 = new Train(obj);
        Train t2 = new Train(obj);
        t1.start();
        t2.start();
    }
}

```

## Output

```

0
1
2
0
1
2

```

## شرح الكود :

الدالة `getline()` هي الدالة التي تحاكي عملية استخدام المورد المشترك هي تقوم بالعد من 0 الى 2 وطباعة الرقم مع ابطاء العملية قليلا

**Synchronized** : عندما يضع خيط مثل `t1` استدعاء `getline()` فانه يحصل على قفل `lock` خاص بكائن الذي تم استدعاء الدالة منه أي خيط آخر مثل `t2` يحاول الوصول الى `getline()` في نفس الوقت سيجد القفل مشغولا ويجبر على الانتظار حتى ينهي الخيط الأول `t1` الدالة بالكامل ويحرر القفل

**كلاس Train** : يرث من كلاس `Thread` هذا يعني ان الكلاس `Train` هو خيط يمكن تشغيله بشكل مستقل

المنشئ يستقبل كائن من النوع `Line` ويخزنه في المتغير `line` هذا يضمن ان الخيطين `t1, t2` يشتركان في نفس نوع الكائن من نوع `Line`

في الدالة الرئيسية يتم انشاء كائن واحد فقط ويتم انشاء خيطين وكلاهما يمرر لهما نفس الكائن ثم بدء تشغيل الخيطين

بسبب استخدام كلمة `Synchronized` ستكون المخرجات منظمة وغير متداخلة

لو تم حذف كلمة `Synchronized` لكانت الخيوط تتنافس بشكل كامل ولكان الخرج متداخل كالآتي 0,0,1,1,2,2

الاتصال بين الخيوط :

- تدعم جافا الاتصال بين الخيوط باستخدام دوال مثل `notifyAll()` `notify()` `wait()`
- هذه الدوال هي جزء من الفئة الرئيسية `Object` في جافا مما يعني ان كل كائن في جافا يمتلكها هذه الدوال تتعامل مع المراقب الخاص بالكائن لاستخدامها بشكل صحيح يجب ان يكون الخيط قد حصل بالفعل على قفل المراقب الخاص بالكائن الذي يستدعي عليه الدالة أي ان استدعاء هذه الدوال يجب ان يتم ضمن كتلة `synchronized` او داخل دالة معرفة باستخدام الكلمة المفتاحية `synchronized` اذا حاولت استدعاء هذه الدوال خارج محيط التزامن ستحصل على خطأ برمجي `illegalMonitorStateException`
- وظيفة الدالة `wait()` : عندما يتم حظر خيط مؤقتا من العمل فانه يستدعي `wait()` هذا يتسبب في نوم الخيط وتحرير المراقب (القفل) الخاص بهذا الكائن مما يسمح لخيط آخر باستخدام الكائن

ملاحظة الدالة `sleep()` لا تحرر قفل

- `notifyAll()` , `Notify()` : تقوم بنقل الخيط النائم من حالة الانتظار الى حالة قابل للتشغيل
- الفرق بين `notifyAll()` , `notify()` :
- `Notify()` : تستأنف عمل خيط واحد تختاره بشكل عشوائي من الخيوط المنتظرة
- `NotifyAll()` : توقف جميع الخيوط النائمة التي تنتظر على قفل هذا الكائن

مثال :

```
public class Account {
    public static int balance = 0;

    public synchronized void withdraw(int amount) {
        while (balance < amount) {
            try {
                System.out.println(Thread.currentThread().getName() + ": Waiting for
deposit...");
                wait();
            } catch (InterruptedException e) {
                System.out.println("Thread Interrupted.");
            }
        }
        balance -= amount;
        System.out.println(Thread.currentThread().getName() + ": Withdrawal successful.
Current balance: " + balance);
    }
}
```

```
public synchronized void deposit(int amount) {
    balance += amount;
    System.out.println(Thread.currentThread().getName() + ": Depositing amount: " +
amount);
    System.out.println(Thread.currentThread().getName() + ": Deposit successful.
Current balance: " + balance);
    notify();
}
}
class WithdrawTask implements Runnable {
    private final Account account;

    public WithdrawTask(Account account) {
        this.account = account;
    }

    public void run() {
        account.withdraw(1000);
    }
}

class DepositTask implements Runnable {
    private final Account account;

    public DepositTask(Account account) {
        this.account = account;
    }
    public void run() {
        try {
            Thread.sleep(2000);
        } catch (InterruptedException e) {}
        account.deposit(2000);
    }
}

public class Main {
    public static void main(String[] args) {
        Account account = new Account();
        Thread threadOne = new Thread(new WithdrawTask(account), "Thread-
Withdraw");
        Thread threadTwo = new Thread(new DepositTask(account), "Thread-Deposit");

        threadOne.start();
        threadTwo.start();
    }
}
```

Expected Output:

Thread-Withdraw: Waiting for deposit...

Thread-Deposit: Depositing amount...

Thread-Deposit: Deposit successful. Current Balance: 2000

Thread-Withdraw: Withdrawal successful. Current Balance: 1000

**شرح مبسط للكود :**

تم وضع الكلمة المفتاحية `synchronized` قبل تعريف الدالتين `deposit()` `withdraw()` هذه الكلمة تضمن ان خيط واحد فقط يمكنه تنفيذ أي من الدوال التي تحمل `synchronized` على نفس الكائن في أي لحظة هذا يمنع مايعرف بسباق البيانات ويضمن سلامة العمليات المالية مثلا لايمكن لخيط A ان يسحب مال بينما الخيط B يقوم بالايذاع في نفس الوقت

اذا كان الرصيد اقل من او يساوي صفر فان الخيط الذي يحاول السحب ينتظر يجب استخدام حلقة `while` وليس `if` لفحص الشرط مرة أخرى بعد الاستيقاظ هذا يضمن ان الخيط لا يستأنف عملية السحب الا اذا كان هناك رصيد كافي بالفعل بعد الخروج من حلقة الانتظار أي عندما يكون الرصيد كافي يتم تحديث الرصيد وتنفيذ عملية السحب

عملية الإيداع بعد ان نجحت عملية الإيداع وتم زيادة الرصيد يتم استدعاء `notify()` لتنبيه خيط واحد من الخيوط المنتظرة التي كانت تحاول السحب بان الرصيد قد تغير

يتم انشاء المهام مهمة السحب ومهمة الإيداع



The End ..